

note

```
>>> import nx_parallel as nxp
>>> H = nxp.ParallelGraph(G)
>>> nx.betweenness centrality(H)
>>> nx.betweenness centrality(G,
backend="parallel")
```

```
$ export NETWORKX_AUTOMATIC_BACKEND=parallel &&
python nx_code.py
```

NOTES:

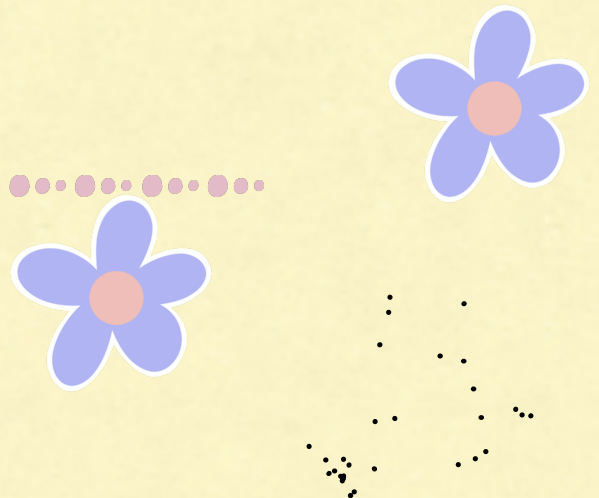
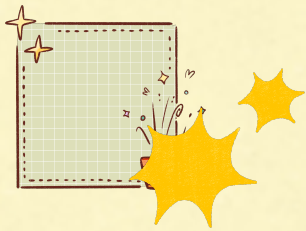
MAKE mistakes



to do



Aditi Tumeja (@Schefflera-Arboreicola)



NetworkX Backend Dispatching



Plugin Arch.



entry_points



Automatic
backend testing

For testing:
NETWORKX_TEST_BACKEND=parallel
NETWORKX_FALLBACK_TO_NX=True
pytest --pyargs networkx "\$@"

9th July, 2024

can_run
should_run
on_start_tests

caching

logging

backend priority

config



NetworkX

A pure Python library for
network analysis

backends

Backend
Interface

backend_info

get_info

"Speed-ups"

local_efficiency-8.8x

betweenness centrality-6.4x

square_clustering-4.1x

bipartite.node_redundancy-4.1x

* for 300-node random graphs (edge probability = 0.6)



too many
parallel
processes
and no
GIL

To chunk or
not to chunk?

nx-parallel

A parallel backend for NetworkX, utilizing Joblib to implement graph
algorithms on multiple CPU cores.

TO-DO

- Designing the pipeline nicely
- Benchmarking
- Adding distributed graph algorithms

feel free to
contribute and
nurture!



Aditi Tumeja (@Schefflera-Arboricola)

Usage:

```
>>> import nx_parallel as nxp
>>> nx.betweenness centrality(G,
>>> backend="parallel")
>>> H = nxp.ParallelGraph(G)
>>> nx.betweenness centrality(H)
```



```
$ export NETWORKX_AUTOMATIC_BACKEND=parallel
&& python nx_code.py
```



Loky



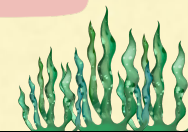
joblib



threading

multiprocessing

dask, ray...



NetworkX Backend Dispatching



Plugin Arch.



entry_points



Automatic
backend testing

For testing:
NETWORKX_TEST_BACKEND=parallel
NETWORKX_FALLBACK_TO_NX=True
pytest --pyargs networkx "\$@"

9th July, 2024



"Speed-ups"

local_efficiency-8.8x
square_clustering-6.7x
betweenness centrality-6.4x
all_pairs_node_connectivity-4.3x
bipartite.node_redundancy-4.1x

can_run
should_run
on_start_tests

caching

logging

backend priority

config

NetworkX

A pure Python library for
network analysis

backends

Backend
Interface

backend_info

get_info

nx-parallel

A parallel backend for NetworkX, utilizing Joblib to implement graph
algorithms on multiple CPU cores.

too many
parallel
processes
and no
GIL

To chunk or
not to chunk?

Usage:

```
>>> import nx_parallel as nxp
>>> nx.betweenness centrality(G,
>>> backend="parallel")
>>> H = nxp.ParallelGraph(G)
>>> nx.betweenness centrality(H)
```

```
$ export NETWORKX_AUTOMATIC_BACKEND=parallel
&& python nx_code.py
```

TO-DO

- Designing the pipeline nicely
- Benchmarking
- Adding distributed graph algorithms

joblib

Loky

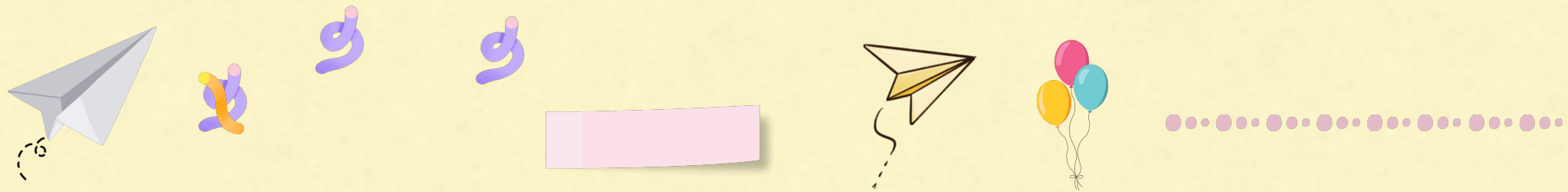
threading

multiprocessing

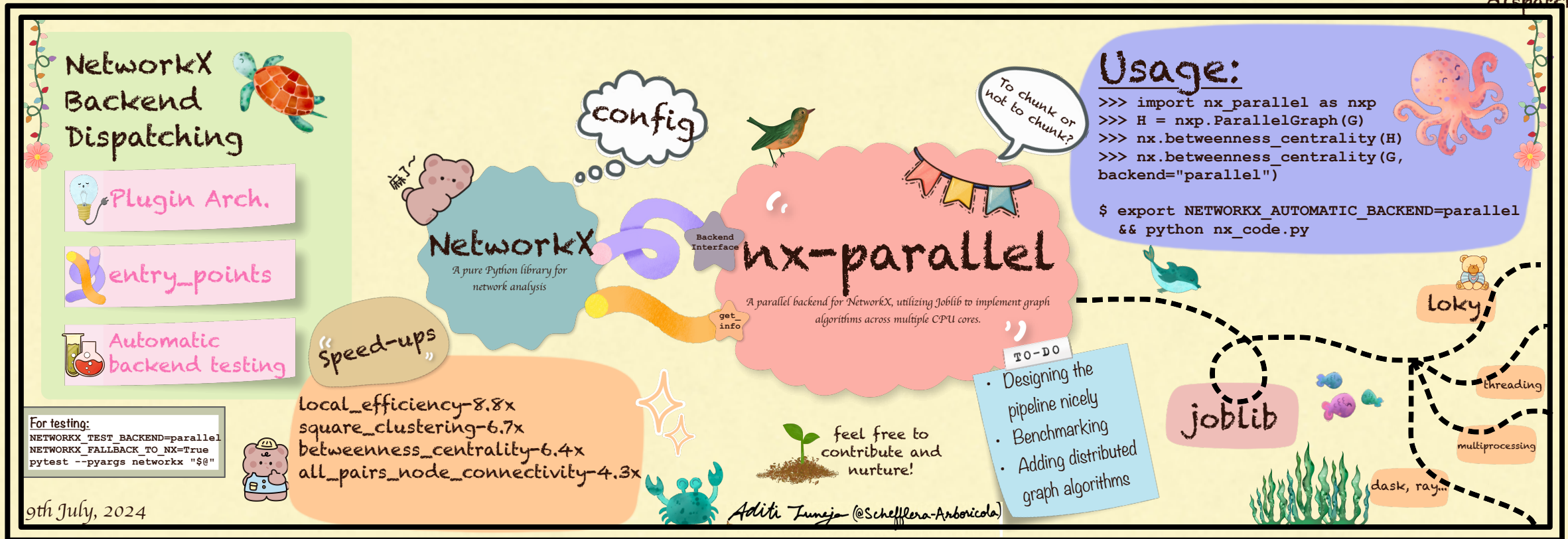
dask, ray...

feel free to
contribute and
nurture!

Aditi Tumeja (@Schefflera-Arboricola)



__networkx_backend
d__ for type based
dispatching





NetworkX Backend Dispatching

 Plugin Arch.

 entry_points

 Automatic backend testing

NetworkX

A pure Python library for network analysis

config

nx-parallel

A parallel backend for NetworkX, utilizing Joblib to implement graph algorithms across multiple CPU cores.

To chunk or not to chunk?

Usage:

```
>>> import nx_parallel as nxp
>>> H = nxp.ParallelGraph(G)
>>> nx.betweenness centrality(H)
>>> nx.betweenness centrality(G, backend="parallel")
```

```
$ export NETWORKX_AUTOMATIC_BACKEND=parallel
&& python nx_code.py
```

to do

- Designing the pipeline nicely
- Benchmarking
- Adding distributed graph algorithms

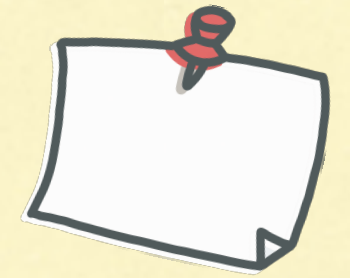
joblib

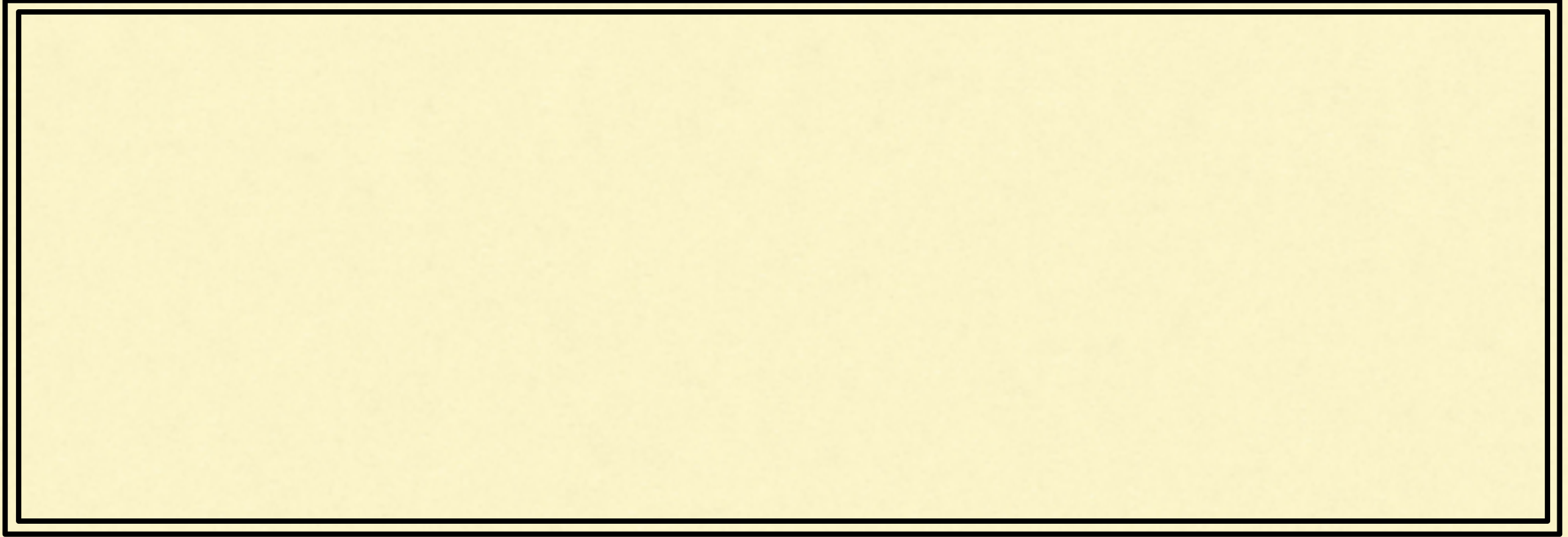
Loky

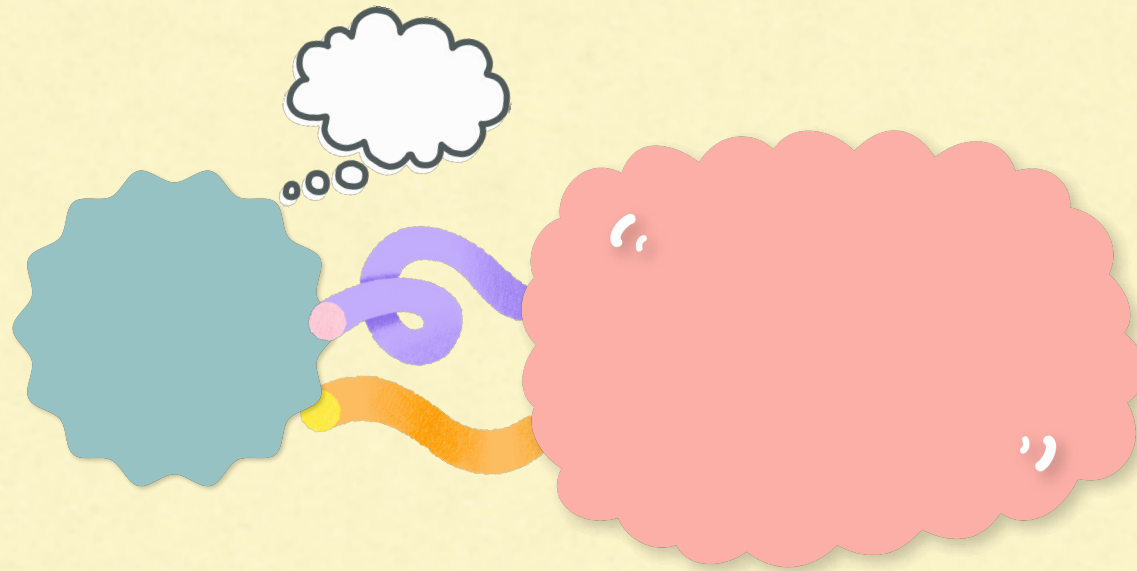
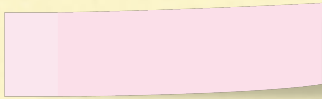
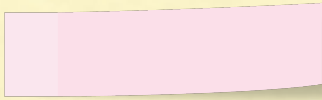
threading

multiprocessing

dask, ray...







to do



Usage:

```
>>> import nx_parallel as nxp
>>> H = nxp.ParallelGraph(G)
>>> nx.betweenness centrality(H)
>>> nx.betweenness centrality(G,
backend="parallel")
```

```
$ export NETWORKX_AUTOMATIC_BACKEND=parallel
&& python nx_code.py
```